Git Workshop

Noah Hallows

2026 Semester 2

What is Git?

Git is a *Distributed Version Control System* created in 2002 by Linus Torvalds to manage the Linux Kernel.

What is Git?

Git is a *Distributed Version Control System* created in 2002 by Linus Torvalds to manage the Linux Kernel.

- A version control system stores the files and folders with a log of their changes. This is called a repository (repo for short).

What is Git?

Git is a *Distributed Version Control System* created in 2002 by Linus Torvalds to manage the Linux Kernel.

- A version control system stores the files and folders with a log of their changes. This is called a repository (repo for short).
- A distributed version control system gives each client a complete copy of this database and files.

What is Git?

Git is a *Distributed Version Control System* created in 2002 by Linus Torvalds to manage the Linux Kernel.

- A version control system stores the files and folders with a log of their changes. This is called a repository (repo for short).
- A distributed version control system gives each client a complete copy of this database and files.

Unlike systems like Google Docs, Git only stores the state of the repository at specific points called **commits**.

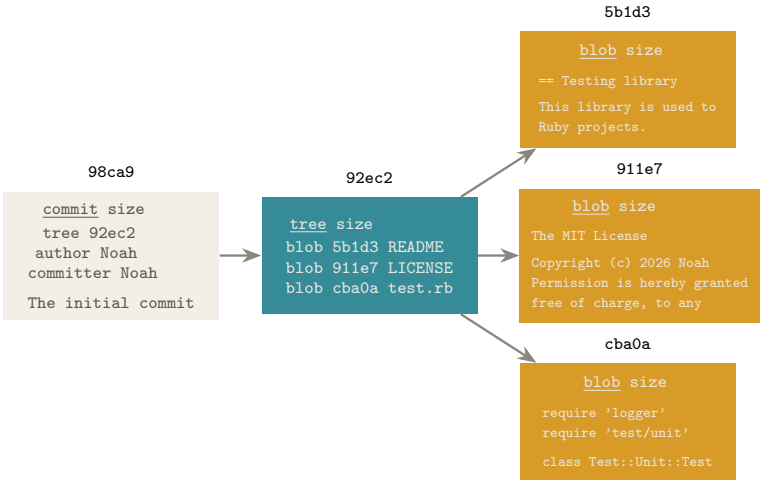
What is Git?

Git is a *Distributed Version Control System* created in 2002 by Linus Torvalds to manage the Linux Kernel.

- A version control system stores the files and folders with a log of their changes. This is called a repository (repo for short).
- A distributed version control system gives each client a complete copy of this database and files.

Unlike systems like Google Docs, Git only stores the state of the repository at specific points called **commits**.
When a commit is created Git takes a snapshot of the repository at that point and refers to it using a sha1 hash.

What is a commit?



Creating a repository

We create a repository using

```
git init
```

Creating a repository

We create a repository using

```
git init
```

This creates a hidden folder called "*.git*" which stores the internal git files

Creating a repository

We create a repository using

```
git init
```

This creates a hidden folder called "*.git*" which stores the internal git files
To "clone" an repository from a remote server we can use

```
git clone https://github.com/cybersecsydney/git-workshop.git
```

Creating a repository

We create a repository using

```
git init
```

This creates a hidden folder called "*.git*" which stores the internal git files
To "clone" an repository from a remote server we can use

```
git clone https://github.com/cybersecsydney/git-workshop.git
```

This gives us a complete local copy of the repository, including the full history
We can also clone using the ssh protocol instead of https

Git identity

Before we can commit anything we need to tell Git who we are.

Git identity

Before we can commit anything we need to tell Git who we are.

```
git config user.name "Noah"
git config user.email "noah@quackmail.com.au"
```

If we don't want to repeat this for each repository we can add the '`--global`' flag.

Git identity

Before we can commit anything we need to tell Git who we are.

```
git config user.name "Noah"
git config user.email "noah@quackmail.com.au"
```

If we don't want to repeat this for each repository we can add the '`--global`' flag.
We can view the repository or global config using

```
git config --edit --global
```

This can be used to store authentication tokens.

Files

Files in a Git repository can be in one of several states.

Files

Files in a Git repository can be in one of several states.

- Untracked

Files

Files in a Git repository can be in one of several states.

- Untracked
- Unmodified

Files

Files in a Git repository can be in one of several states.

- Untracked
- Unmodified
- Modified

Files

Files in a Git repository can be in one of several states.

- Untracked
- Unmodified
- Modified
- Staged

Files

Files in a Git repository can be in one of several states.

- Untracked
- Unmodified
- Modified
- Staged

When we first create a file it is untracked
Before we can commit a file we need to *stage* it for next commit by adding it to the *staging area*

```
git add filename.txt
```

Files

Files in a Git repository can be in one of several states.

- Untracked
- Unmodified
- Modified
- Staged

When we first create a file it is untracked
Before we can commit a file we need to *stage* it for next commit by adding it to the *staging area*

```
git add filename.txt
```

"git add" can be thought of as the "add this to the next commit" command.

Files

Files in a Git repository can be in one of several states.

- Untracked
- Unmodified
- Modified
- Staged

When we first create a file it is untracked
Before we can commit a file we need to *stage* it for next commit by adding it to the *staging area*

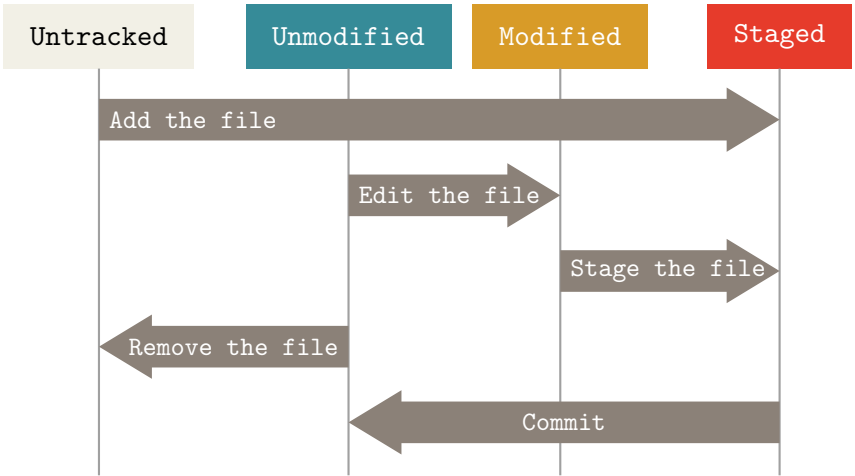
```
git add filename.txt
```

"git add" can be thought of as the "add this to the next commit" command.

Warning

Don't commit environment files

File areas


Untracked
Unmodified
Modified
Staged
Add the file
Edit the file
Stage the file
Remove the file
Commit

Files cont.

Before we commit we should inspect the staging area using:

```
git status
```

Files cont.

Before we commit we should inspect the staging area using:

```
git status
```

Once we are happy with the staging area we commit it

```
git commit
```

Files cont.

Before we commit we should inspect the staging area using:

```
git status
```

Once we are happy with the staging area we commit it

```
git commit
```

There are numerous useful options such as

- `-m <msg>`
- `-S`

Once we've made this commit we can push it to the remote repo using:

```
git push
```

Remark

Filenames in ".gitignore" are automatically ignored by git.

Removing files

Similarly to add we can remove files from the staging area.

```
git rm --cached filename.txt
```

Removing files

Similarly to add we can remove files from the staging area.

```
git rm --cached filename.txt
```

Or if we want to untrack and remove a file

```
git rm filename.txt
```

Removing files

Similarly to add we can remove files from the staging area.

```
git rm --cached filename.txt
```

Or if we want to untrack and remove a file

```
git rm filename.txt
```

Remark

This doesn't remove the file from the history.

Commit messages

When writing a commit message make it informative so people can easily see what the commit is about.

Commit messages

When writing a commit message make it informative so people can easily see what the commit is about.

Figure: Randall Munroe, https://xkcd.com/1296

Commit messages

When writing a commit message make it informative so people can easily see what the commit is about.

Figure: Randall Munroe, https://xkcd.com/1296

Warning

Some classes will mark you down for uninformative commit messages

History

To view previous commits run

```
git log
```

History

To view previous commits run

```
git log
```

Commits are referred to by a sha1 hash, for example

```
d05e2ac13494daf33d26004d52f577bea19c9e52
```

There are various options to modify the output format, including `--pretty`. It is displayed using the Unix command *less*

Diff(erence)

`git diff`

This command is used to show the line by line difference between any two commits, or the staging area, or the working directory.

Diff(erence)

`git diff`

This command is used to show the line by line difference between any two commits, or the staging area, or the working directory.
To see the difference between the latest commit on two different branches

```
git diff main some-branch
```

Diff(erence)

`git diff`

This command is used to show the line by line difference between any two commits, or the staging area, or the working directory.
To see the difference between the latest commit on two different branches

```
git diff main some-branch
```

To see the difference between the current working directory and a commit

```
git diff HEAD
```

Diff(erence)

`git diff`

This command is used to show the line by line difference between any two commits, or the staging area, or the working directory.
To see the difference between the latest commit on two different branches

```
git diff main some-branch
```

To see the difference between the current working directory and a commit

```
git diff HEAD
```

Remark

HEAD is a pointer the latest commit on the current branch

Rewriting history

There are multiple ways to undo things in Git, but some of these cannot be undone so you may lose work.
There are four main commands used

Rewriting history

There are multiple ways to undo things in Git, but some of these cannot be undone so you may lose work.
There are four main commands used

- `git reset`

Rewriting history

There are multiple ways to undo things in Git, but some of these cannot be undone so you may lose work.
There are four main commands used

- `git reset`
- `git revert`

Rewriting history

There are multiple ways to undo things in Git, but some of these cannot be undone so you may lose work.
There are four main commands used

- `git reset`
- `git revert`
- `git commit --amend`

Rewriting history

There are multiple ways to undo things in Git, but some of these cannot be undone so you may lose work.
There are four main commands used

- `git reset`
- `git revert`
- `git commit --amend`
- `git checkout`

Rewriting history

There are multiple ways to undo things in Git, but some of these cannot be undone so you may lose work.
There are four main commands used

- `git reset`
- `git revert`
- `git commit --amend`
- `git checkout`

Remark

Some classes will mark you down if you include junk files in your repository, this is how you will remove them

git reset

```
git reset <commit>
```

This is used to change the commit which HEAD points to, allowing it to be used to undo commits amount other things.

git reset

`git reset <commit>`

This is used to change the commit which HEAD points to, allowing it to be used to undo commits amount other things. It has three main modes:

- `--soft`. This leaves your working directory unchanged and keeps staged changes.

git reset

```
git reset <commit>
```

This is used to change the commit which HEAD points to, allowing it to be used to undo commits amount other things. It has three main modes:

- `--soft`. This leaves your working directory unchanged and keeps staged changes.
- `--mixed`. This leaves your working directory unchanged but unstages changes.

git reset

`git reset <commit>`

This is used to change the commit which HEAD points to, allowing it to be used to undo commits amount other things. It has three main modes:

- `--soft`. This leaves your working directory unchanged and keeps staged changes.
- `--mixed`. This leaves your working directory unchanged but unstages changes.
- `--hard`. This overwrites your working directory and unstages changes, **meaning you lose any uncommitted changes**.

git reset

```
git reset <commit>
```

This is used to change the commit which HEAD points to, allowing it to be used to undo commits amount other things. It has three main modes:

- `--soft`. This leaves your working directory unchanged and keeps staged changes.
- `--mixed`. This leaves your working directory unchanged but unstages changes.
- `--hard`. This overwrites your working directory and unstages changes, **meaning you lose any uncommitted changes**.

```
git reset HEAD~5
```

git revert

Git revert makes a new commit that undo the changes made by a individual commit *without deleting* that commit. The `--no-commit` flag is used to put the changes in the staging area instead of automatically creating a commit.

commit –amand

This command is used to modify the most recent commit, allowing the commit message to be changed and files to be added. However it will completely overwrite the old commit.

git checkout

The git checkout command is used to change branches and restore files in the working tree.

git checkout

The git checkout command is used to change branches and restore files in the working tree.

- Switching branches

git checkout

The git checkout command is used to change branches and restore files in the working tree.

- Switching branches
- Creating branches

git checkout

The git checkout command is used to change branches and restore files in the working tree.

- Switching branches
- Creating branches
- Navigating to a specific commit

git checkout

The git checkout command is used to change branches and restore files in the working tree.

- Switching branches
- Creating branches
- Navigating to a specific commit
- Restore or revert changes in the working tree

git checkout

The git checkout command is used to change branches and restore files in the working tree.

- Switching branches
- Creating branches
- Navigating to a specific commit
- Restore or revert changes in the working tree

To restore a specific file to the last committed version:

```
git checkout --file.txt
```

git checkout

The git checkout command is used to change branches and restore files in the working tree.

- Switching branches
- Creating branches
- Navigating to a specific commit
- Restore or revert changes in the working tree

To restore a specific file to the last committed version:

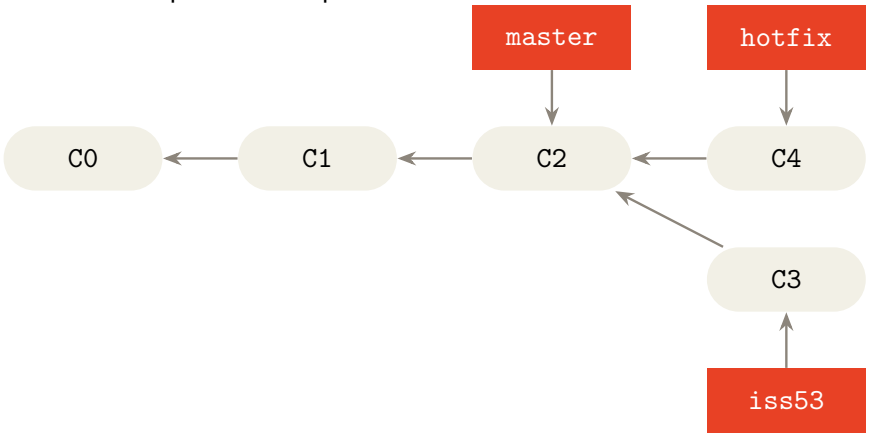
```
git checkout --file.txt
```

Remark

A detached HEAD means you are not on any branch

Branches

Branches are pointers to specific commits.



Merging

Say we want to merge branches iss53 and hotfix to apply the hotfix to our iss53 branch. We would first switch to the iss53 branch (using git checkout iss53) and then run:

```
git merge hotfix
```

Merging

Say we want to merge branches iss53 and hotfix to apply the hotfix to our iss53 branch. We would first switch to the iss53 branch (using git checkout iss53) and then run:

```
git merge hotfix
```

If there are merge conflicts we will see an error. To resolve these conflicts we need to manually modify the conflicting files.

Merging

Say we want to merge branches iss53 and hotfix to apply the hotfix to our iss53 branch. We would first switch to the iss53 branch (using git checkout iss53) and then run:

```
git merge hotfix
```

If there are merge conflicts we will see an error. To resolve these conflicts we need to manually modify the conflicting files.
Once the conflicts are resolved stage the changes and continue the merge using:

```
git merge hotfix --continue
```

PEP

Thanks to the book Pro Git (second edition) by Scott Chacon and Ben Straub.
Thanks to the Honorable Rudransh Kala for something.